

# Stack, Pointer & Assembly

Cepet Ngerti, Langsung Lanjut



# LIST OF CONTENTS

Gimana Stack Bekerja & Stack Frame

Stack Tumbuh Ke Bawah

Pointer & Memory di C

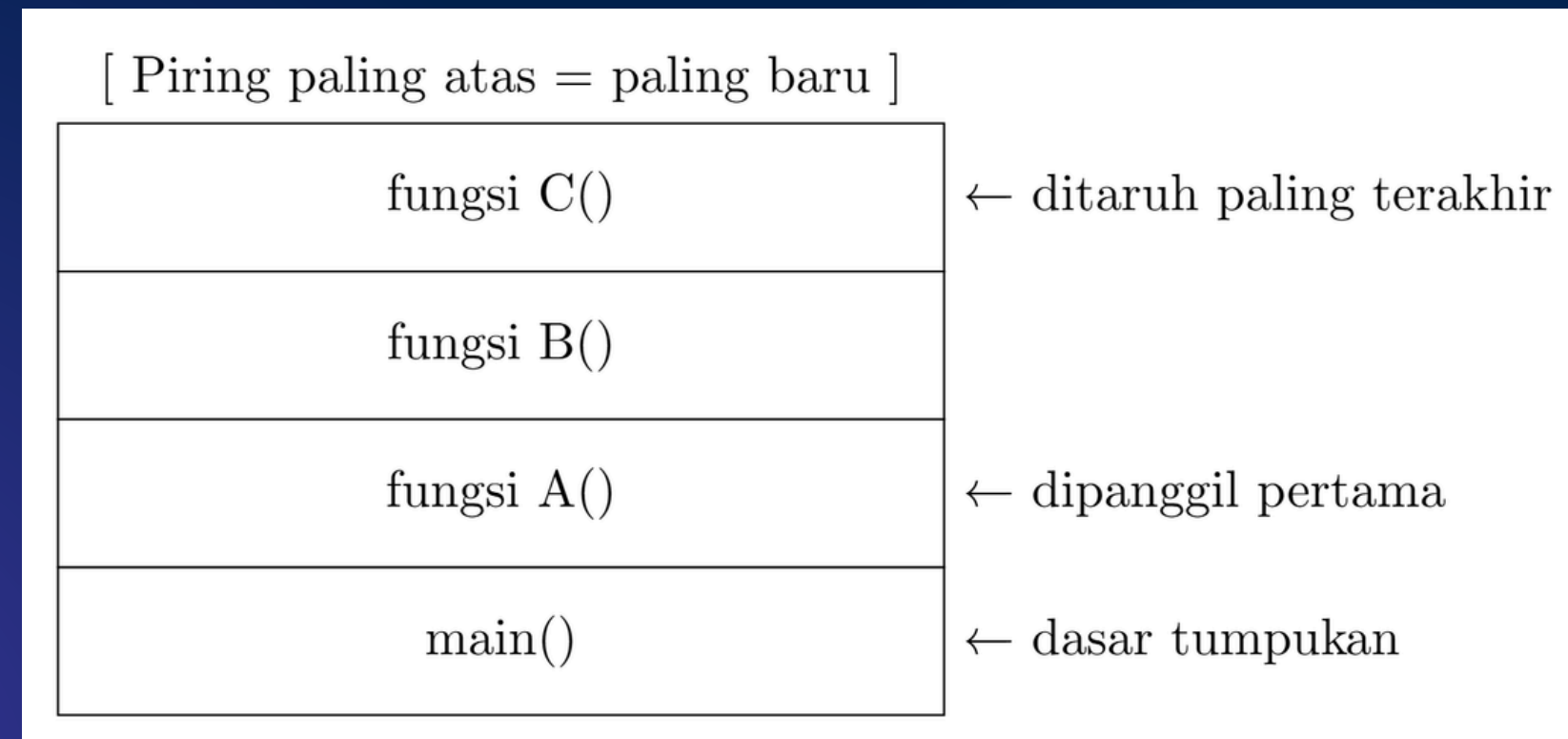
Stack & Pointer, Hubungannya Apa?

Assembly x86-64 & Register Penting

call dan ret Itu Kuncinya

Rangkuman





## Gimana Stack Bekerja?

Stack itu kayak tumpukan piring

Aturannya simpel:

- Masuk → PUSH (tumpuk ke atas)
- Keluar → POP (ambil dari atas)
- Istilahnya: LIFO — Last In, First Out



|                                          |                                        |
|------------------------------------------|----------------------------------------|
| return address                           | → balik ke mana setelah fungsi selesai |
| saved RBP                                | → simpan posisi frame sebelumnya       |
| local variables<br>char buf[64]<br>int x | → variabel lokal fungsi ini            |

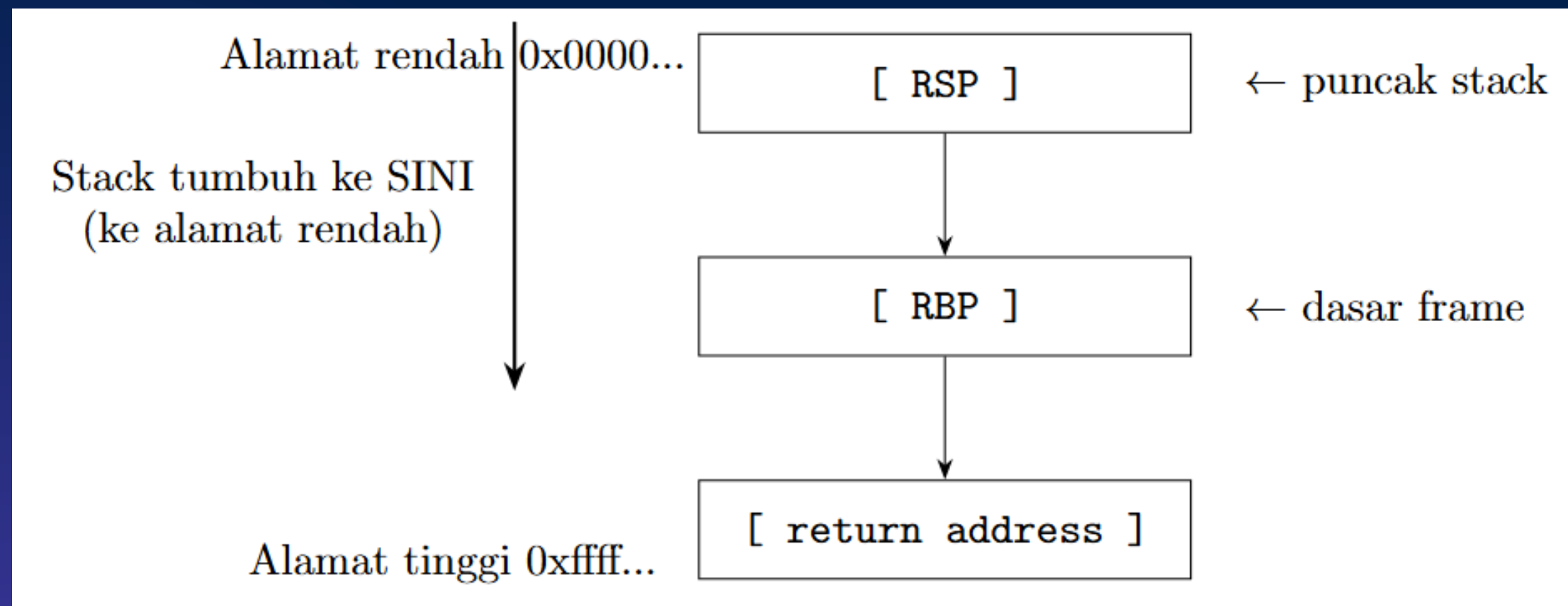
## Gimana Stack Bekerja?

Tiap fungsi yang dipanggil bikin stack frame sendiri:

2 register penting:

- RSP → nunjuk ke puncak stack (Stack Pointer)
- RBP → nunjuk ke dasar frame ini (Base Pointer)





## Stack itu Tumbuh Kebawah

Stack di x86-64 dimulai dari alamat tinggi dan tumbuh ke arah alamat rendah. Artinya setiap kali PUSH dieksekusi, RSP dikurangi 8 (bukan ditambah), lalu nilai disimpan di alamat baru itu. Sebaliknya, POP akan menambah RSP sebesar 8. Jadi "puncak" stack bukan di alamat tertinggi — justru di alamat terendah yang aktif saat itu, yaitu di mana RSP nunjuk.





|          |       |       |       |       |       |
|----------|-------|-------|-------|-------|-------|
| Alamat → | 0x100 | 0x101 | 0x102 | 0x103 | 0x104 |
| Isi →    | 41    | 42    | 43    | 00    | ??    |

↑  
 "ABC\0"

## memory

array raksasa bernomor

```

int x = 42;
int *p = &x;    // p menyimpan ALAMAT x, bukan nilai x

printf("%d", x);    // → 42      (nilai langsung)
printf("%p", p);    // → 0x7fff... (alamat x)
printf("%d", *p);   // → 42      (dereference: ambil nilai
dari alamat)
  
```

## pointer

variabel yang menyimpan alamat

# Memory vs Pointer







```
char nama[5] = "Andi";
```

```
// Ini semua sama aja:
```

```
nama[0] == 'A'
```

```
*(nama + 0) == 'A'
```

```
*nama == 'A'
```

```
nama[1] == 'n'
```

```
*(nama + 1) == 'n'
```

Array name = pointer ke elemen pertama



```
char buf[64];
```

```
// buf == &buf[0] == alamat awal buffer
```

Makanya waktu buffer overflow, kita nulis mulai dari buf[0] terus ke atas sampai nabrak return address

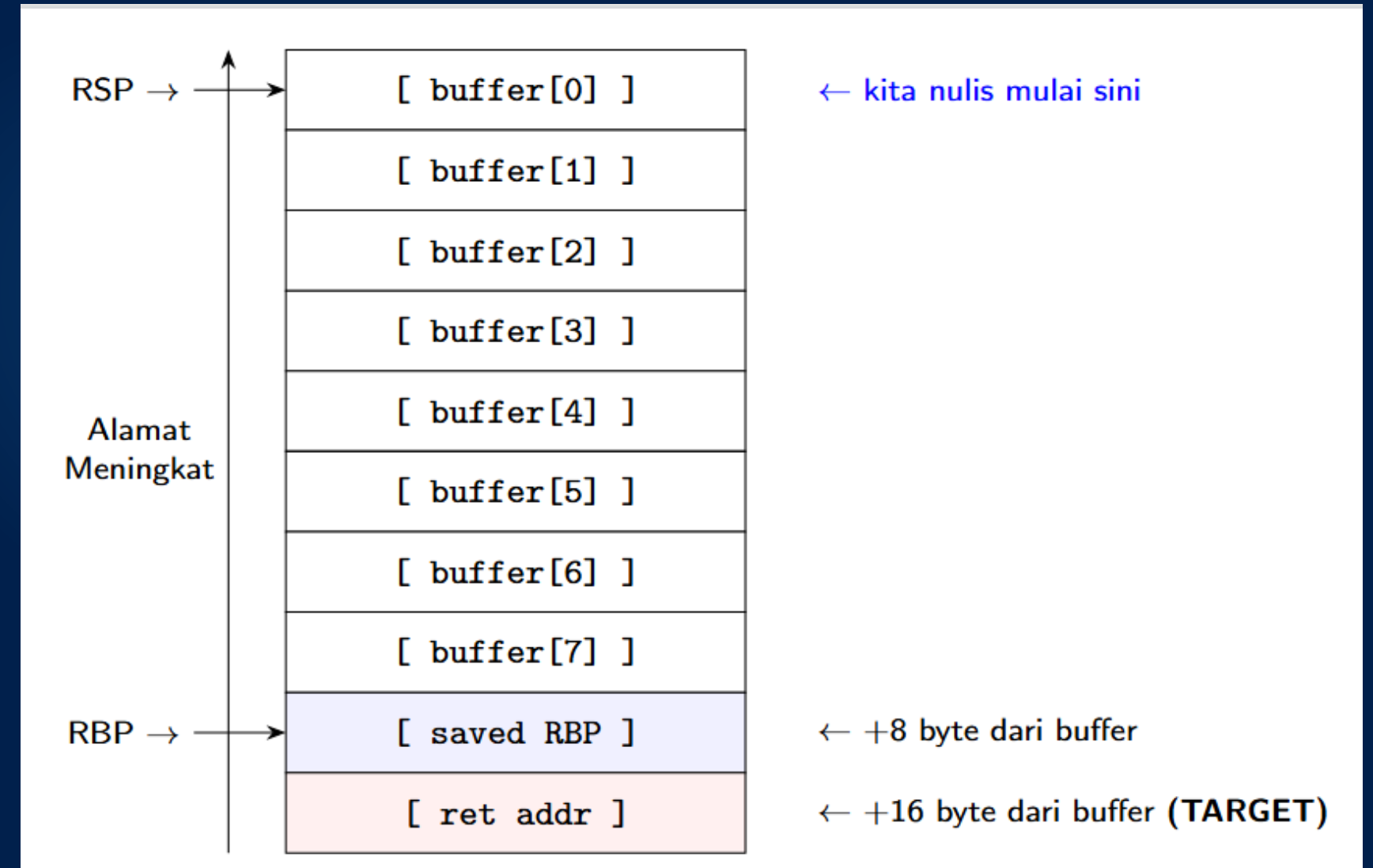
## Pointer & Array, Saudara Kandung





```
void login() {  
    char buffer[8]; // di-alokasi di stack  
    gets(buffer);   // nulis ke alamat buffer  
}
```

Di memory waktu login() dipanggil:



## Stack & Pointer Nyambung Gimana?







```
mov rax, 5      ; Isi register RAX dengan nilai 5
push rax        ; Simpan (push) nilai RAX ke dalam stack
pop rax         ; Ambil (pop) nilai dari stack, simpan kembali ke RAX

call fungsi     ; Simpan return address ke stack dan lompat ke label 'fungsi'
ret             ; Ambil return address dari stack dan lompat ke alamat tersebut

add rax, 1       ; Tambahkan 1 ke register RAX (RAX = RAX + 1)
cmp rax, rbx     ; Bandingkan nilai di RAX dengan nilai di RBX
jmp 0x401196     ; Lompat (jump) secara absolut ke alamat memori 0x401196
```

## Assembly x86-64, Yang Penting Aja

Lo gak perlu hafal semua instruksi. Cukup tau yang ini:



|     |                                                                          |
|-----|--------------------------------------------------------------------------|
| RAX | → Return value fungsi / hasil operasi                                    |
| RBX | → General purpose                                                        |
| RCX | → Counter (loop)                                                         |
| RDX | → Data / Argument ke-3                                                   |
| RSI | → Argument ke-2                                                          |
| RDI | → Argument ke-1 [PENTING UNTUK EXPLOIT]                                  |
| RSP | → Stack Pointer [PENTING UNTUK EXPLOIT]                                  |
| RBP | → Base Pointer                                                           |
| RIP | → Instruction Pointer (Program Counter) ← <b>TARGET UTAMA CORRUPTION</b> |

ALUR EKSEKUSI

## Register Yang Sering Muncul

RIP = "program lagi eksekusi instruksi di mana" Kalau lo bisa kontrol RIP = lo kontrol programnya



### Waktu call fungsi() dieksekusi:

1. CPU push alamat instruksi

BERIKUTNYA ke stack

(ini yang namanya return address)

2. CPU lompat ke alamat fungsi

### Waktu ret dieksekusi:

1. CPU pop nilai dari puncak stack

2. CPU lompat ke alamat itu

### Kesimpulan:

ret = pop RIP

Kalau kita kontrol nilai di puncak stack saat ret... Kita kontrol ke mana program lompat Dan itu persis yang kita lakuin di buffer overflow

## call dan ret Itu Kuncinya

RIP = "program lagi eksekusi instruksi di mana" Kalau lo bisa kontrol RIP = lo kontrol programnya



```
gdb ./vuln
(gdb) disas login
```

```
Dump of assembler code for function login:
0x00401162 <+0>:  push    rbp
0x00401163 <+1>:  mov     rbp, rsp
0x00401166 <+4>:  sub     rsp, 0x48      ← alokasi buffer 72 byte
0x0040116a <+8>:  lea     rax, [rbp-0x40] ← alamat buffer
0x0040116e <+12>: mov     rdi, rax
0x00401171 <+15>: call    0x401040 <gets@plt> ← panggil gets()
0x00401176 <+20>:  nop
0x00401177 <+21>:  leave
0x00401178 <+22>:  ret                  ← di sinilah exploit kita kerja
```

## Baca Assembly Di GDB

sub rsp, 0x48 → buffer ada di 72 byte dari RSP  
Offset ke return address = 72 + 8 (saved RBP) = 80  
byte



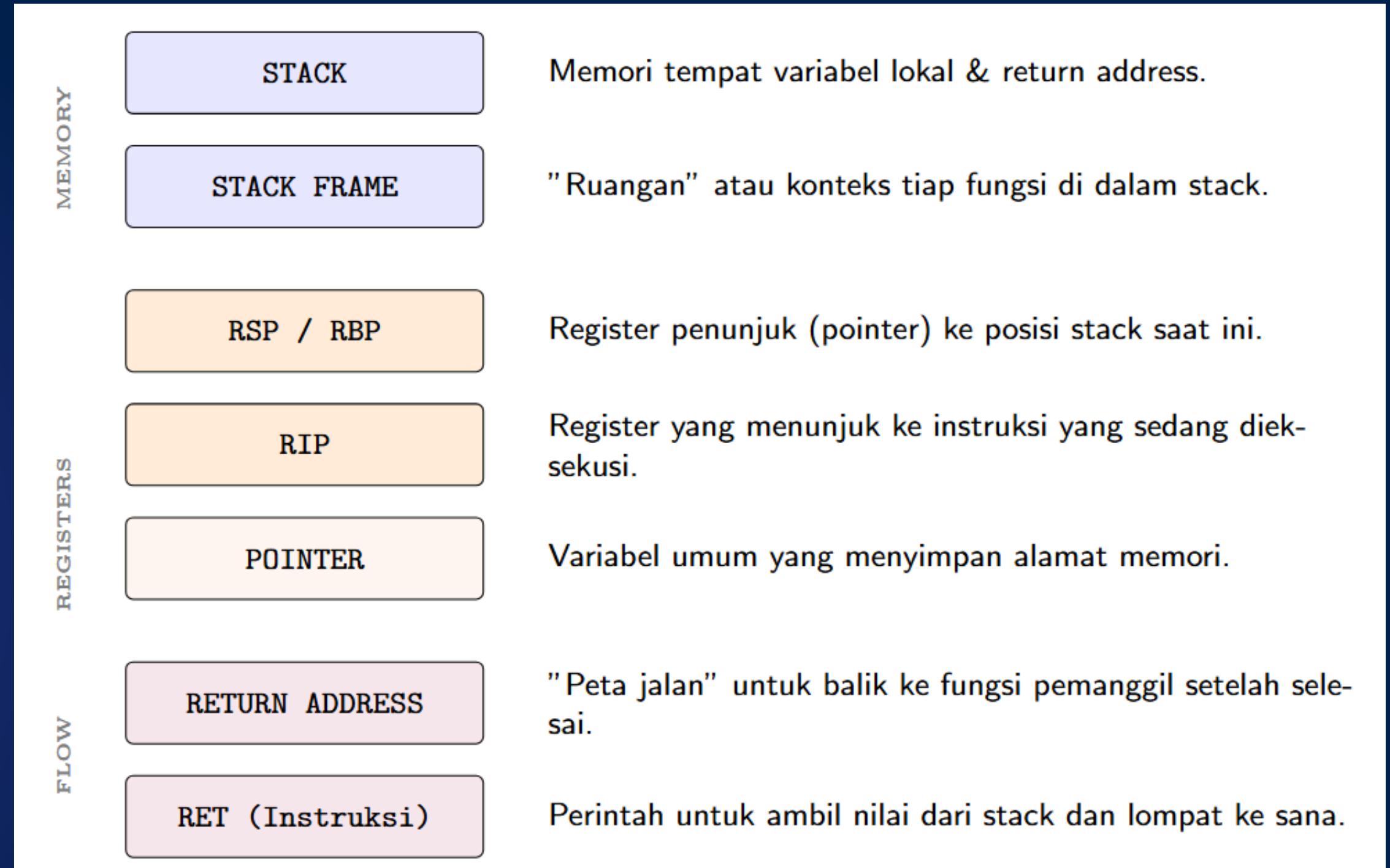


# RECAP

## Hubungannya ke Buffer

### Overflow:

Buffer ada di stack → Kalau kita nulis kebanyakan → Return address ketimpa → ret lompat ke alamat kita → Game over buat programnya



Thank You  
**Thank You**  
Thank You